Quantum-Assisted DirectCentauro Reconstruction

Local Integration Report for the Electron-Ion Collider

Outcome. A public, source-only local Aer and sibling EICrecon integration is delivered. Accepted progress is recorded by hash-bound external ledgers: shadow 7080/7080, hard 600/600, and end-to-end 40/40 are complete; active comparison is 56/300 and partial. No final aggregate is available.

Public technical report

Abstract

This report documents a bounded local integration for DirectCentauro jet reconstruction. C++ retains event state, candidate construction, clustering actions, and output production. A local Python service samples a candidate index and returns it through a strict JSONL contract. The contract binds a response to the exact request bytes with SHA-256 before active-mode use. The delivered repository includes the Python implementation, copy-ready sibling EICrecon additions, integration instructions, protocol tests, and this report. The evidence supports implementation and progress-coverage claims only; it does not establish a performance result or quantum advantage.

Contents

1 Outcome and Scope

MEASURED The deliverable is a source-inspected implementation of a local candidate selector for DirectCentauro reconstruction, accompanied by a sibling EICrecon integration package and inspection documentation. The public repository is organized so that a reviewer can identify the ownership boundary, protocol contract, and evidence limits without reconstructing the full development history.

Table 1: Report status at delivery.

Area	Status	Meaning
Delivered source	Complete	Local Python selector, C++ additions, wrappers, tests, and documentation are bundled.
Integration boundary	Complete	The exchange is local AF_UNIX JSONL with strict request and reply validation.
Evidence coverage	Partial	Accepted progress is recorded by hash-bound external ledgers, not aggregate physics or performance statistics.
Overall assessment	PASS WITH LIMITATIONS	The active comparison stopped before its planned coverage and has no final aggregate.

The unpublished internship brief supplied by the author and supervisor provides motivation and initial deliverable context [1]; it is not a bundled public source. By local agreement, portions outside the delivered implementation were excluded from final public scope. This report therefore describes only the delivered local implementation and its documented evidence.

OUT OF SCOPE This report does not claim quantum speedup, production readiness, universal FastJet equivalence, full active statistics, or hardware results.

2 Physics Context and DirectCentauro

Jet reconstruction groups particles into physically meaningful final-state objects. In deep-inelastic scattering, the choice of coordinates and distance measure matters because the beam direction is distinguished. Centauro is an asymmetric jet-clustering context developed for this setting [3]. The DirectCentauro implementation here constructs pair and beam candidates directly in the reconstruction algorithm rather than presenting itself as a replacement for another jet package.

For each active working jet, the implementation derives a Centauro-coordinate representation from its four-vector. Pair candidates use a non-negative distance in that coordinate space, while a beam candidate finalizes an active jet. The classical selector chooses the first finite minimum, which makes equal-distance behavior deterministic.

One clustering iteration

C++ builds ordered pair and beam candidates from active jets. A selection identifies either a pair to merge or one jet to finalize. C++ applies that action and repeats until no active jets remain.

Figure 1: One clustering iteration

MEASURED The direct implementation creates candidates, applies merges or beam actions, preserves constituent references, and writes DirectCentauro jet output in the sibling integration additions. It is intentionally an experimental direct implementation. Published jet-algorithm references motivate the context; they are not validation evidence for this implementation [4, 3].

3 Delivered Source

The repository separates reusable local service code from the copy-ready additions required by a compatible sibling EICrecon checkout. The source tree is the primary public artifact; this report does not require private paths, protected outputs, or generated campaign files.

Table 2: Review map for the delivered source.

Path	Purpose
`src/eic_quantum/contracts/`	Strict truth-blind request and response schema plus digest validation.
`src/eic_quantum/payloads/`	Inverse-power state preparation for bounded candidate lists.
`src/eic_quantum/services/`	Persistent local Aer worker and bounded error handling.
`integration/eicrecon/additions/`	C++ reconstruction, selector, socket client, factory, and focused test additions.
`integration/eicrecon/README.md`	Exact copy list and two required sibling changes.
`tests/python/`	Unit coverage for the exact-wire request digest.
`docs/`	Architecture, method, and status boundaries.

The Python material and copied C++ additions retain their documented licensing and provenance in the repository. The integration guide identifies nine additions and two tracked sibling modifications. This keeps the public deliverable reviewable without asserting that the additions have been accepted upstream.

MEASURED The requested output contract names the event header, reconstructed Breit-frame particles, and reconstructed DirectCentauro jets. The exact collection identifiers are listed in the public integration guide. It preserves the event header, immediate Breit constituents, and DirectCentauro jets; it is not an archival event export.

4 C++/Python Architecture and Exact-Wire Contract

The architecture has a narrow ownership boundary. C++ owns authoritative reconstruction state: particle input, candidate distances, clustering mutations, classical fallback, and output collections. Python's only actionable reconstruction proposal is the bounded sampling index; its response diagnostics also include counts, probabilities, timings, and metadata. The service cannot receive evaluation truth such as a classical minimum or final partition in its strict selector request.

Local ownership boundary

C++ DirectCentauro builds an ordered candidate list, sends one strict JSON request to the local worker, validates the response, and decides whether it may be applied. Python receives only ordered candidates and execution controls, samples a bounded index, and returns contract metadata.

Figure 2: Local ownership boundary

The service exchange uses one JSON object per AF_UNIX socket line. Both sides validate required fields, candidate indexing, finite non-negative distances, response shape, and bounds. A response includes `request_sha256`. The worker hashes the exact UTF-8 payload it received, excluding the JSONL line-feed delimiter; C++ hashes the exact serialized request bytes it sent and compares the values before accepting the index.

Table 3: Fail-closed response checks.

Check	Consequence in active mode
Malformed schema or fields	Reply is rejected.
Digest or request identifier mismatch	Reply is rejected.
Index outside the sent candidate list	Reply is rejected.
Unavailable service or timeout	Reconstruction fails closed.
Valid bounded reply	Only then may its candidate index be applied.

MEASURED A historical scoped safe check recorded exactly seven focused pytest tests passing locally. This is local unit verification, not campaign measurement; notebook source inspection is architecture evidence, not protocol validation. The contract is a correctness and traceability mechanism, not a performance measurement.

5 Local Selector Method

For a positive finite candidate-distance list d_i, the local method prepares amplitudes proportional to d_i^{-a}. Consequently, its ideal sampling probabilities are proportional to d_i^{-2a}. The default controls in the delivered wrappers are exponent $a = 3.0$, 512 shots, seed 314159, and no more than 128 candidates.

$$p_i = \frac{d_i^{-2a}}{\sum_j d_j^{-2a}}.$$

The implementation normalizes in the log domain, removes only numerically negligible probability mass according to its documented cutoff, pads to a power-of-two state dimension, and prepares a measured circuit for local Aer sampling. An exact-zero pair distance uses a deliberate local C++ bypass: no worker request or digest is created and the exact-zero candidate is applied.

PARTIAL Finite-shot sampling is not an exact minimum finder. The returned candidate is the highest-count sampled index with deterministic index tie handling. This method is motivated by quantum-clustering literature [2]; that reference does not establish an Electron-Ion Collider performance result.

The bounded candidate cap is part of the contract. Oversized candidate lists, timeouts, and invalid replies do not silently become a different problem: their outcome depends on the configured fail-closed policy. The bare configuration defaults to classical fallback; the public active wrapper forces fail closed, while the public shadow wrapper defaults fail closed with an exploratory false override.

6 Sibling EICrecon Integration and Modes

The public integration is designed for a compatible sibling EICrecon checkout. The integration guide specifies the additions to copy, the two build-registration changes to apply, and the runtime variables required by the local worker and reconstruction wrappers. No modification to a sibling checkout is performed by this report build.

Table 4: Selection semantics.

Mode	Behavior
Classical	C++ applies the deterministic first finite minimum.
`qiskit_shadow`	A local selection is requested and recorded, while C++ applies the classical action.
`qiskit_active`	C++ applies a local index only after strict validation and exact-wire digest agreement.

Shadow mode separates observation from action: it records what the local selector proposed while preserving the classical clustering transition. Active mode changes that final selection step only when the reply is valid. The bare EICrecon configuration defaults `quantumFailClosed=false` and may classically fall back. The public active wrapper forces `true`; the public shadow wrapper defaults to `true` and offers only an exploratory `false` override. Oversize, timeout, and invalid-reply behavior depends on that configuration. An exact-zero pair distance is applied locally by C++ without a worker request or digest.

MEASURED The integration retains C++ ownership throughout both modes. The worker does not construct event data, mutate jets, or write EDM output. This division is essential for interpreting a selector comparison: it is a comparison of candidate-choice behavior within a fixed reconstruction loop, not a replacement of the reconstruction framework.

7 Evidence and Current Status

The following counts are accepted progress recorded by hash-bound external ledgers. Accepted means rows admitted only after validation. Raw ledgers are not bundled, so public repository contents alone cannot independently reproduce every count. These are coverage records, not numerical summaries of physics quality, agreement, latency, or scaling.

Table 5: Supplied progress coverage.

Coverage	Status	Interpretation
Shadow	7080/7080 complete	The shadow workflow completed its recorded coverage.
Hard	600/600 complete	The hard-case workflow completed its recorded coverage.
End-to-end	40/40 complete	The end-to-end workflow completed its recorded coverage.
Active	56/300 partial	The active comparison stopped before full planned coverage.
Final aggregate	Absent	No aggregate result is reported.

PARTIAL The active comparison stopped with the classification `comparison_layer_type_error`: no exact constituent-partition match left `max()` with no delta values, causing an unadmitted `TypeError`. Earlier accepted rows remain separate. This prevents treating the partial active count as a full comparison or inferring an aggregate conclusion.

MEASURED The overall assessment is **PASS WITH LIMITATIONS**. It means source-inspected code and the listed hash-bound progress are documented, while the active comparison remains incomplete. No fabricated numerical summary is introduced here.

Evidence interpretation

Completed counts establish that their listed coverage completed. The partial active count and absent aggregate limit the conclusion to implementation status and bounded progress, not performance or equivalence.

Figure 3: Evidence interpretation

8 Limitations and Conclusion

The public conclusion is deliberately narrow. The project delivers a source-inspected local selector integration with an explicit C++/Python ownership boundary, strict request and response validation, and exact-wire digest binding. It also delivers integration instructions and a transparent record of hash-bound accepted and incomplete progress coverage.

Table 6: Interpretation limits.

Not established	Why it is not established here
Quantum speedup	No controlled asymptotic or end-to-end performance evidence is reported.
Full active comparison	Only 56 of 300 active records are available, and the comparison layer ended with a type error.
Universal FastJet equivalence	This is a direct experimental implementation with bounded evidence, not a universal equivalence proof.
Production readiness	The delivery is source-only and integration-oriented, with stated limitations.
Hardware results	The delivered scope is local Aer only.

For maintainers, the next technically meaningful work is to repair and independently re-run the comparison layer before creating any final aggregate. That work must preserve the same separation between selector proposal, classical evaluation, and reconstruction ownership. Until then, the correct public statement remains **PASS WITH LIMITATIONS**.

MEASURED The report is self-contained with respect to public relative repository paths. It cites the original plan as context [1] and distinguishes motivation from delivered evidence.

Campaign vocabulary is intentionally narrow: accepted means a row admitted only after validation, and progress means accepted progress recorded by hash-bound external ledgers. The raw ledgers are not bundled, so every count is not independently reproducible from this public repository alone. The active failure occurred when no exact constituent-partition match left `max()` with no delta values, causing an unadmitted `comparison_layer_type_error`; earlier accepted rows remain separate.

References

[1] Unpublished internship brief. *Quantum Algorithms at the Electron-Ion Collider*. Unpublished internship brief supplied by the author and supervisor, 2026; not a bundled public source.

[2] Alvaro Martinez de Lejarza, Leandro Cieri, and German Rodrigo. Quantum clustering and jet reconstruction at the LHC. *Physical Review D*, 106:036021, 2022. doi:10.1103/PhysRevD.106.036021.

[3] Miguel Arratia, Yiannis Makris, Duff Neill, Felix Ringer, and Nobuo Sato. Asymmetric jet clustering in deep-inelastic scattering. *Physical Review D*, 104:034005, 2021. doi:10.1103/PhysRevD.104.034005.

[4] Matteo Cacciari, Gavin P. Salam, and Gregory Soyez. FastJet user manual. *European Physical Journal C*, 72:1896, 2012. doi:10.1140/epjc/s10052-012-1896-2.